

◆ 1. What is an Observable in Angular?

An **Observable** is a stream of data over time. Instead of returning a single value (like a function), it can emit:

- Multiple values
- Over time (async)
- Lazily (only when subscribed)

👉 Think of it as a **data pipeline**.

Example Concept:

- Button clicks → stream
 - HTTP response → stream
 - Form value changes → stream
-

◆ 2. Why Angular Uses RxJS?

Angular internally integrates **RxJS** for:

- Async operations (HTTP calls)
- Event handling
- State changes
- Reactive forms

Key Advantages:

- Handles async cleanly
 - Supports operators (map, filter, debounce)
 - Enables reactive programming model
-

◆ 3. Observable vs Promise

Feature	Observable	Promise
Values	Multiple	Single
Lazy	Yes	No
Cancelable	Yes (unsubscribe)	No
Operators	Rich (RxJS)	Limited
Use Case	Streams	One-time async

◆ 4. Creating an Observable

Basic Example

```
import { Observable } from 'rxjs';

const myObservable = new Observable(observer => {
  observer.next('Hello');
  observer.next('Angular');
  observer.complete();
});
```

◆ 5. Subscribing to Observable

```
myObservable.subscribe({
  next: (value) => console.log(value),
  error: (err) => console.error(err),
  complete: () => console.log('Done')
});
```

👉 Without subscribe(), nothing executes (lazy execution).

◆ 6. Angular HTTP Example (Real-time)

Angular's **HttpClient** returns Observables:

```
import { HttpClient } from '@angular/common/http';
```

```
constructor(private http: HttpClient) {}
```

```
getUsers() {  
  return this.http.get('https://api.example.com/users');  
}
```

Component:

```
this.getUsers().subscribe(data => {  
  console.log(data);  
});
```

◆ 7. RxJS Operators (Core Strength)

Operators transform streams.

Example:

```
import { map } from 'rxjs/operators';
```

```
this.http.get('api/users')  
  .pipe(  
    map((res: any) => res.data)  
  )  
  .subscribe(data => console.log(data));
```

◆ 8. Common RxJS Operators

Transformation

- `map()`
- `switchMap()`
- `mergeMap()`

Filtering

- `filter()`
- `debounceTime()`

Utility

- `tap()` (side effects)
 - `catchError()`
-

◆ 9. Angular 20+ Modern Pattern (Signals + RxJS)

Angular 20+ introduces **Signals**, but Observables are still widely used.

👉 Interop example:

```
import { toSignal } from '@angular/core/rxjs-interop';
```

```
usersSignal = toSignal(this.http.get('/api/users'));
```

👉 This converts Observable → Signal

◆ 14. Async Pipe (Best Practice)

Avoid manual subscribe:

```
<div *ngIf="users$ | async as users">
```

```
  {{ users | json }}
```

</div>

👉 Angular handles subscription automatically

◆ 15. Summary (Key Takeaways)

- Observables = async data streams
 - RxJS = powerful library to control streams
 - Angular heavily depends on RxJS
 - Operators = core power
 - Async pipe = cleanest approach
 - Signals are new, but RxJS is still essential
-

◆ When to Use What?

Scenario	Use
HTTP calls	Observable
Events	Observable
Complex async flows	RxJS
Simple state	Signals